zerotrading-core — Architecture Plan

Status: Planning pass only. No implementation code yet.
Launch strategy: New-hour fast-entry / fast-exit on Kalshi KXBTCD hourly markets.
Launch scope: Single open position at a time.

1. Reference Repo Audit (`kalshi-btcd-trader`)

Preserve as reusable core infrastructure

Module	What to keep	Notes
`server/kalshi.ts`	RSA-PSS auth logic (`createSignature`, `kalshiRequest`)	Correct, battle-tested. Minor reshape: extract to `adapters/kalshi/auth.ts`.
`server/kalshi.ts`	`getOrderbook` + `estimateSlippage`	Excellent NO-centric bid/ask inversion with full depth walk. Keep verbatim.
`server/kalshi.ts`	`getKxbtcdMarkets` + pagination guard + CURRENT_HOUR filter	The pagination fix (PR #4 guard) is hard-won; preserve it exactly.
`server/kalshi.ts`	`placeOrder` + `cancelOrder` (limit-order cross pattern)	The Kalshi quirk of always sending a price field even for "market" orders must be preserved.
`server/kalshi.ts`	`getBtcPrice` (Coinbase primary / Binance fallback)	Solid dual-source. Move to `adapters/btc-price.ts`.
`server/marketData.ts`	`fetchExternalBtcData` + `fetchBinanceFunding`	BinanceVision endpoint trick (no geo-block). Cache-TTL logic is correct. Move to `adapters/market-data.ts`.
`server/marketData.ts`	`computeRealizedVol`, `computeMomentum` math	Pure functions; extract to `core/market-math.ts`.
`server/storage.ts`	`IStorage` interface	Keep as the persistence contract verbatim.
`server/tradingEngine.ts`	`CircuitBreakerState` + `BreakerCause` enum + `armBreaker` logic	Well-structured. Extract to `core/circuit-breaker.ts`.
`server/tradingEngine.ts`	`SettlementAttribution` shape	Good audit trail. Keep in `core/types.ts`.
`server/tradingEngine.ts`	`settleClosedTrade` dual-source attribution (Kalshi confirmed vs. estimated)	Critical for accounting correctness. Port to `core/accounting.ts`.

Module	What to keep	Notes
`server/tradingEngine.ts`	Persistent circuit breaker file (`CB_STATE_FILE`) survive-restart pattern	Important for cooldown continuity across Railway restarts. Port to `persistence/`.
`server/tradingEngine.ts`	`OrderbookCacheEntry` / `orderbookCache` TTL map	Move to `core/orderbook-cache.ts`.
`shared/schema.ts`	`trades` table shape and `SettingsLog` concept	Use as data model reference.
`tests/`	`touchProb.spec.ts`, `terminalProb.spec.ts`	Keep test harness design, re-run against new math.

Discard entirely

Module / Pattern	Why
`client/` (entire React app + all `ui/` components)	UI is explicitly deferred. Start headless.
`server/vite.ts`, `server/static.ts`	Vite dev-server coupling is launch-irrelevant.
`server/routes.ts` REST API surface	Overly exposed for launch; replace with a minimal status endpoint.
`server/polymarket.ts`	Cross-venue signal is strategy logic, not core infra. Deferred.
`server/deribit.ts`	Same — optional enrichment signal. Deferred.
`shared/schema.ts` → `signals` table	RSI/MACD-flavored signal log is a relic of an older strategy shape. Not needed at launch.
`botSettings` schema (entire 40-field table)	Settings sprawl is the explicit anti-pattern to avoid. Replace with a small `LaunchConfig`.
`legacyGenerateSignal` / `legacy_fixed_band` mode	The old mode is superseded. Don't carry dead weight into the new repo.
`strategyMode` toggle	Two-mode flag was technical debt from an in-flight migration. zerotrading-core has one strategy interface.
Global `state` singleton object	Mutable singleton is the root cause of reconciliation bugs in the old engine. Replace with explicit message-passing between engine steps.
`decaySignal` / `currentMarket` (legacy UI compat fields)	Backward-compat cruft. Gone.

Rewrite from scratch

What	Why rewrite
`tradingEngine.ts` main loop	2855-line monolith with mixed concerns: accounting, risk, execution, strategy, and UI broadcast all in one file. Separate into layer-bounded modules.
`storage.ts` implementation class	File-write-on-every-call works but has race conditions under fast cycles. Replace with a proper write-behind queue and a clean adapter that slots to Supabase later.
`schema.ts` config model	Needs to start at ~8 fields for launch, not 40. Define `LaunchConfig` as a plain TypeScript object with Zod validation, not a DB table.
Circuit breaker auto-resume + `cleanCycleRequired` flow	Logic is correct but interleaved into the main loop. Extract as a standalone state machine.
Accounting / net liq computation (`calcNetLiq`)	Currently inlined into main loop. Needs to be a pure function with a typed input surface, testable independently.

2. Clean Architecture: `zerotrading-core`

Design axioms

1. **Accounting is the source of truth.** The engine never trusts its own in-memory state for P&L; it reconciles against the exchange on every cycle.
2. **Each layer has one job.** Layers communicate through typed interfaces, not shared mutable state.
3. **The strategy is a plugin.** The engine core knows nothing about KXBTCD specifics — it calls a `StrategyAgent` interface.
4. **Config is a typed, validated object.** No database table for settings at launch. One file, one Zod schema, minimal fields.
5. **Persistence is swap-out.** The `IEventLog` / `IPositionStore` interfaces start as JSON-file implementations, upgrade to Supabase without touching the engine.

3. Folder Structure

```
zerotrading-core/
│
├── src/
│   │
│   ├── adapters/                    # Exchange + data source I/O only. No logic.
│   │   ├── kalshi/
│   │   │   ├── auth.ts              # RSA-PSS signing, kalshiRequest wrapper
│   │   │   ├── market-feed.ts        # getKxbtcdMarkets, getOrderbook, estimateSlip
│   │   │   ├── execution.ts          # placeOrder, cancelOrder
│   │   │   └── portfolio.ts          # getBalance, getOpenPositions, getSettledPosi
│   │   ├── btc-price.ts              # getBtcPrice (Coinbase + Binance fallback)
```

```
│   │   └── market-data.ts           # fetchExternalBtcData, fetchBinanceFunding (Bir
│   │
│   ├── core/                        # Pure business logic. No I/O, no side effects.
│   │   ├── types.ts                 # Shared domain types: Position, Fill, OrderRe
│   │   ├── market-math.ts           # computeRealizedVol, computeMomentum, compute
│   │   ├── accounting.ts            # calcNetLiq, reconcileFill, attributeSettlemer
│   │   ├── circuit-breaker.ts       # CircuitBreaker state machine: armBreaker, ch
│   │   └── orderbook-cache.ts       # TTL-keyed orderbook cache with explicit inval
│   │
│   ├── engine/                      # Orchestration. Wires adapters + core together
│   │   ├── runner.ts                # setInterval loop, lifecycle: start/stop/pause
│   │   ├── cycle.ts                 # Single-cycle execution: fetch → reconcile → ri
│   │   ├── reconciler.ts            # Fill confirmation, order expiry, position re
│   │   ├── risk.ts                  # Circuit breaker evaluation, position sizing (
│   │   └── execution-policy.ts      # chooseExecutionMode, computeEntryLimitPrice,
│   │
│   ├── strategy/                    # Strategy interface + hour-harvester implement
│   │   ├── interface.ts             # StrategyAgent interface: evaluate(context): S
│   │   ├── context.ts               # CycleContext shape passed into every strategy
│   │   └── hour-harvester/
│   │       └── index.ts             # Launch strategy: placeholder until math is imp
│   │
│   ├── persistence/                 # Storage implementations. Swap file ↔ Supabas
│   │   ├── interfaces.ts            # IPositionStore, IEventLog, ICircuitBreakerSt
│   │   ├── file/
│   │   │   ├── position-store.ts    # JSON file: open trades, last known position
│   │   │   └── event-log.ts         # Append-only NDJSON event log (one line per ev
│   │   └── supabase/                # (stub) — implement later without changing engi
│   │       ├── position-store.ts
│   │       └── event-log.ts
│   │
│   ├── config/
│   │   ├── schema.ts                # Zod schema for LaunchConfig (intentionally sm
│   │   ├── defaults.ts              # Default values for all config fields
│   │   └── loader.ts                # Load from env + .config.json, validate, freez
│   │
│   └── index.ts                     # Entry point: load config → init storage → star
│
├── tests/
│   ├── accounting.spec.ts
│   ├── circuit-breaker.spec.ts
│   ├── market-math.spec.ts
│   └── reconciler.spec.ts
│
├── .config.json.example             # Config template (tracked, no secrets)
├── package.json
├── tsconfig.json
└── railway.json
```

4. Layer Responsibilities

`adapters/`

- **Only responsibility:** Talk to the outside world. Transform raw API responses into domain types.
- Must not contain any business logic, scoring, or decision-making.
- Every function is async and returns either a typed result or throws a typed error.
- `kalshi/auth.ts` — RSA-PSS signing and authenticated request wrapper. No knowledge of what is being fetched.
- `kalshi/market-feed.ts` — Pagination-safe KXBTCD fetch (preserves the PR #4 guard), NO-centric orderbook normalization.
- `kalshi/execution.ts` — placeOrder with the required limit-price-even-for-market-orders Kalshi quirk, cancelOrder.
- `kalshi/portfolio.ts` — Balance, open positions, settled positions, order fills. Used exclusively by the reconciler.
- `btc-price.ts` — Dual-source price feed. Returns `{ price, source, fetchedAt }`.
- `market-data.ts` — Binance klines + funding. TTL cache lives here, not in the engine.

`core/types.ts`

Single source of truth for shared domain types:

```
Position         // open trade: ticker, side, count, entryPriceCents, strikePrice,
Fill             // confirmed fill: orderId, count, avgPriceCents, filledAt
OrderResult      // from placeOrder: orderId, status, ticker, side, count, priceCe
SettlementAttribution  // full exit record (preserved from old repo)
CycleContext     // what the engine hands to the strategy each cycle (see strategy/c
```

No logic — just types and enums.

`core/accounting.ts`

Pure functions. No I/O. Fully unit-testable.

- `reconcileFill(order, fills) → Fill | null` — determines if an order is filled from fill records
- `calcPnl(position, exitPriceCents, feeRate) → PnlResult` — computes gross P&L and fee estimate
- `attributeSettlement(position, kalshiSettlement, btcPrice) → SettlementAttribution` — dual-source attribution (Kalshi confirmed vs. estimated). Ported from `settleClosedTrade`.
- `calcNetLiq(balance, position, marketQuote) → number`

`core/circuit-breaker.ts`

Explicit state machine. No I/O. Testable in isolation.

- `CircuitBreakerState` type
- `checkBreakers(state, pnlResult, riskParams) → CircuitBreakerState` — evaluates all triggers after every exit
- `checkAutoResume(state, now) → CircuitBreakerState` — called at cycle start; auto-resumes if cooldown expired
- `resetHourlyCounters(state, now) → CircuitBreakerState` — pure, returns new state

The file-based persist/restore for Railway restarts is a side effect that lives in `persistence/file/` and is called by the engine, not by this module.

`engine/cycle.ts`

The main per-cycle orchestrator. Calls adapters and core in a fixed order:

```
1. Fetch BTC price
2. Fetch external market data (candles, funding) — cached
3. Fetch KXBTCD markets
4. Fetch balance + open positions (Kalshi)
5. Reconcile fill status of any pending order (→ reconciler.ts)
6. If position exists and market closed: attributeSettlement, update event log
7. Evaluate circuit breakers (→ risk.ts)
8. If no active position AND breaker clear: call StrategyAgent.evaluate(context)
9. If strategy returns signal: evaluate execution policy, place order (→ adapters/ka
10. Log CycleEvent to event log
```

No business logic for *what* to trade — that is entirely the strategy's concern.

`engine/reconciler.ts`

Handles the order lifecycle from "placed" to "filled or cancelled":

- Check if pending order is confirmed via open positions (fast path)
- If not filled within TTL: fetch fills, cancel if empty, update event log
- On fill: update position store, log `FillEvent`

`engine/risk.ts`

Reads `RiskParams` from config. Evaluates whether the engine should accept a strategy signal:

- Position sizing: `computeContractCount(balance, riskParams) → number`
- Pre-trade checks: min balance, max open risk, circuit breaker active, time filters
- Returns `{ approved: boolean, reason: string }`

The circuit breaker is evaluated here but the state machine lives in `core/circuit-breaker.ts`.

`strategy/interface.ts`

```
interface StrategyAgent {
  name: string;
  evaluate(ctx: CycleContext): Promise<TradeSignal | null>;
}

interface TradeSignal {
  ticker: string;
  side: "yes" | "no";
  action: "buy";
  rationale: string;           // logged verbatim to event log
  maxEntryPriceCents: number;  // strategy's price ceiling; execution may use tight
}
```

The engine does not know what scoring, probability, or edge models the strategy uses. It only consumes `TradeSignal | null`.

`strategy/context.ts`

```
interface CycleContext {
  btcPrice: number;
  markets: KalshiMarket[];          // current-hour markets only (already filtered)
  orderbooks: Map<string, KalshiOrderbook>;  // pre-fetched by engine for top candi
  externalData: BtcExternalData;
  balance: number;
  approvedContractCount: number;     // from risk.ts — strategy must not exceed this
  cycleAt: number;                   // epoch ms
}
```

`strategy/hour-harvester/`

Launch strategy. Scope:

- Receives `CycleContext`
- Selects a candidate from `ctx.markets`
- Returns a `TradeSignal` or `null`
- Contains all strategy-specific math (to be implemented in a later pass)

At launch, this file will be a stub that always returns `null` until the strategy math is wired in.

`persistence/interfaces.ts`

```
interface IPositionStore {
  getActivePosition(): Promise<Position | null>;
  setActivePosition(p: Position): Promise<void>;
  clearActivePosition(): Promise<void>;
}

interface IEventLog {
```

```
  append(event: LogEvent): Promise<void>;
  tail(n: number): Promise<LogEvent[]>;
}

interface ICircuitBreakerStore {
  load(): Promise<PersistedBreakerState | null>;
  save(s: PersistedBreakerState): Promise<void>;
  clear(): Promise<void>;
}
```

File implementation at launch. `persistence/supabase/` stubs added now so the interface contract is proven; implementations come later. Swapping persistence back-ends must never require changes above `persistence/`.

`config/schema.ts`

The intentionally small launch config surface (see section 5 below).

5. Minimal Launch Config Surface

Config is loaded from environment variables + an optional `.config.json`, validated with Zod, and frozen at startup. There is no settings database table.

```
// LaunchConfig — everything the engine needs at launch
interface LaunchConfig {
  // Exchange credentials
  kalshiApiKeyId: string;         // from env: KALSHI_API_KEY_ID
  kalshiPrivateKeyPem: string;    // from env: KALSHI_PRIVATE_KEY_PEM
  kalshiEnv: "production" | "demo";  // default: "production"

  // Engine control
  pollIntervalSeconds: number;    // default: 30
  enabled: boolean;               // master on/off switch; default: false

  // Position sizing
  riskPerTradePct: number;        // % of balance risked per trade; default: 5

  // Execution guardrails
  maxSpreadCents: number;         // reject if NO spread > this; default: 8
  minDepthContracts: number;      // reject if depth at ask < this; default: 5
  maxSlippageCents: number;       // reject if estimated slippage > this; default:

  // Circuit breakers
  maxDailyLossUsd: number;        // hard daily stop; default: 50
  maxConsecutiveLosses: number;   // loss streak cooldown trigger; default: 3
  cooldownMinutes: number;        // cooldown duration on loss streak; default: 60
}
```

8 non-credential fields total. Every field has a sensible default so the minimal viable config is just credentials + `enabled: true`.

Intentionally excluded at launch:

- Strategy tuning parameters (minEdgeBps, rewardRiskRatio, zone preferences) — move in when strategy math is implemented
- Cross-venue signals (Polymarket, Deribit) — deferred
- Exit-policy fine-tuning (stopLossConfirmCycles, cheapZoneHold) — deferred

6. Persistence Design

Launch: file-based

- `data/position.json` — single active position (or empty)
- `data/events.ndjson` — append-only newline-delimited JSON event log, one line per event
- `data/circuit-breaker.json` — persisted breaker state for Railway restart continuity

Event types logged to `events.ndjson`:

```
CycleEvent      { ts, btcPrice, regime, decision, reason }
OrderPlacedEvent { ts, orderId, ticker, side, count, priceCents, rationale }
FillEvent       { ts, orderId, ticker, avgFillCents, count }
ExitEvent       { ts, ticker, exitPriceCents, pnlDollars, exitReason }
SettlementEvent  { ts, attribution: SettlementAttribution }
BreakerEvent    { ts, cause, reason, cooldownUntilMs }
```

This log is the canonical audit trail. The position store is just a cache derived from it.

Future: Supabase

- `IEventLog` implementation posts rows to `events` table
- `IPositionStore` reads/writes to `positions` table
- No engine code changes — only swap in `persistence/supabase/` implementations in `index.ts`

7. What Is In Scope for Launch

- Adapters: Kalshi auth, market feed, execution, portfolio
- BTC price + Binance klines (momentum + vol)
- Single-position accounting and reconciliation
- Circuit breaker state machine (daily loss + loss streak + cooldown)
- File persistence: position store + NDJSON event log + circuit breaker persist/restore
- Execution policy: `cross_if_edge_survives` and `join_best_price` modes
- Hour-harvester strategy stub (returns null until math is added)
- Minimal 8-field config with Zod validation
- Unit tests for accounting, circuit breaker, market math, reconciler

- Railway deployment (`railway.json`, health endpoint — single route `/health` only)

8. Deferred (Not Scope for Launch)

Item	Rationale
Strategy math (probability model, edge scoring, candidate ranking)	Explicit deferral per constraints. Stub interface is in scope.
UI / dashboard	Lowest priority. NDJSON log is sufficient for monitoring.
REST API surface	No public API at launch except `/health`.
Polymarket signal adapter	Deferred cross-venue signal.
Deribit signal adapter	Deferred cross-venue signal.
Multiple concurrent strategies	Architecture supports it via `StrategyAgent[]`; single strategy at launch.
Supabase persistence	Interfaces proven at launch; implementation deferred.
SSE broadcast / WebSocket live feed	Only needed when UI exists.
Settings hot-reload	Config is frozen at startup; restart required to change.
Backtesting harness	Useful but not launch-blocking.
Volatility/momentum filter tuning params in config	Added to config only when strategy math is wired.

9. Recommended Build Order

Build in this sequence. Each step is deployable and testable before the next begins.

```
Step 1 — Types + Config
  src/core/types.ts
  src/config/schema.ts + defaults.ts + loader.ts
  ✓ Test: Zod validation of LaunchConfig, env loading

Step 2 — Kalshi Adapter
  src/adapters/kalshi/auth.ts        (port createSignature + kalshiRequest)
  src/adapters/kalshi/portfolio.ts   (getBalance, getOpenPositions, getSettledPosit
  src/adapters/kalshi/market-feed.ts (getKxbtcdMarkets, getOrderbook, estimateSlipp
  src/adapters/kalshi/execution.ts   (placeOrder, cancelOrder)
  src/adapters/btc-price.ts
  src/adapters/market-data.ts
  ✓ Test: connect to demo, fetch markets, fetch balance

Step 3 — Core Logic (pure, testable)
  src/core/market-math.ts           (computeRealizedVol, computeMomentum, computeR
  src/core/accounting.ts            (reconcileFill, calcPnl, attributeSettlement, ca
  src/core/circuit-breaker.ts       (state machine only, no I/O)
  src/core/orderbook-cache.ts
  ✓ Tests: accounting.spec, circuit-breaker.spec, market-math.spec
```

```
Step 4 — Persistence Layer
  src/persistence/interfaces.ts
  src/persistence/file/position-store.ts
  src/persistence/file/event-log.ts
  src/persistence/file/circuit-breaker-store.ts  (persist/restore from disk for Rai
  ✓ Test: write → restart → read continuity

Step 5 — Strategy Interface + Stub
  src/strategy/interface.ts
  src/strategy/context.ts
  src/strategy/hour-harvester/index.ts  (stub — returns null always)

Step 6 — Engine
  src/engine/reconciler.ts          (fill confirmation, order expiry logic)
  src/engine/risk.ts                (position sizing, circuit breaker evaluation, pr
  src/engine/execution-policy.ts    (chooseExecutionMode, computeEntryLimitPrice)
  src/engine/cycle.ts               (orchestrates steps 1-10, calls strategy stub)
  src/engine/runner.ts              (setInterval lifecycle, start/stop)
  src/index.ts                      (wire everything, /health endpoint)
  ✓ Test: full dry-run cycle in demo mode, engine runs N cycles, no position opened

Step 7 — Strategy Math (next planning pass)
  src/strategy/hour-harvester/index.ts  (replace stub with actual implementation)
  src/config/schema.ts                  (add strategy tuning params to LaunchConfig)
```

Key Decisions Summary

Decision	Rationale
No global mutable `state` singleton	The old repo's root cause of subtle reconciliation bugs. Engine state flows forward explicitly through typed function parameters.
`StrategyAgent` interface from day one	Enforces that the engine core is strategy-agnostic. Adding a second strategy later is additive, not a refactor.
NDJSON event log (not trade table only)	Every decision is auditable: why the engine did not trade is as important as why it did.
Config as frozen object, not DB table	Eliminates the 40-field settings sprawl. Config changes require a deploy, which is intentional — config is not a runtime tuning knob.
File persistence first, Supabase second	Ship sooner. The `IEventLog`/`IPositionStore` interfaces are the migration path.
No UI scope at launch	A headless engine that logs correctly is more valuable than a dashboard that masks accounting bugs.
Accounting correctness before strategy math	Step 3 (pure accounting functions + tests) must be green before Step 6 (engine) is built. Never trust the engine's P&L reporting until the accounting layer has independent test coverage.